

ChatLLM - guia rapido de implementacao

Instrucoes personalizadas por usuario | FastAPI + SQLite + React

Objetivo deste PDF

Servir como mapa de raciocinio e consulta rapida. Ele mostra onde mexer, o que cada parte precisa fazer, a ordem recomendada, estruturas genericas de sintaxe e testes. Nao traz uma implementacao pronta para copiar integralmente.

Repositorio analisado: [viccsneto/chatllm_experiment_without_ai](https://github.com/viccsneto/chatllm_experiment_without_ai) (branch master)

Preparado em 30/09/2026

1. O que a feature precisa fazer

A funcionalidade e equivalente a Custom Instructions: cada usuario autenticado pode salvar um texto proprio, recuperar esse texto depois de novo login e fazer com que esse texto seja usado como system prompt nas conversas. Se nunca tiver salvo nada, o comportamento atual deve continuar exatamente igual.

- Persistir o texto por usuario no SQLite.
- Permitir consultar e atualizar apenas as instrucoes do usuario autenticado.
- Usar as instrucoes tanto na resposta normal quanto no streaming.
- Manter o `_SYSTEM_PROMPT` atual como fallback.
- Manter isolamento: usuario A nao pode ler ou alterar dados do usuario B.
- Expor uma interface simples no React para visualizar, editar e salvar.

Armadilha importante do SQLite

O projeto chama `Base.metadata.create_all(bind=engine)`. Isso cria tabelas que ainda nao existem, mas nao adiciona automaticamente uma coluna nova a uma tabela `users` que ja existe. Por isso, para uma execucao sem migration, uma tabela separada ligada 1:1 ao usuario e uma opcao robusta.

2. Mapa do repositorio

Arquivo	Papel na feature
backend/models.py	Modelos SQLAlchemy. Aqui entram as estruturas persistentes.
backend/services/openrouter.py	Contem <code>_SYSTEM_PROMPT</code> , <code>_build_messages</code> , <code>generate_reply</code> e <code>stream_reply</code> .
backend/routers/chat.py	Recebe o usuario autenticado e chama o OpenRouter. Ha caminho normal e streaming.
backend/routers/	Lugar natural para um router autenticado de configuracoes/instrucoes.
backend/schemas/	Modelos Pydantic de request/response.
backend/main.py	Registra os routers com <code>app.include_router(...)</code> .
frontend/src/api.js	Centraliza chamadas HTTP e ja injeta Authorization Bearer.
frontend/src/App.jsx	Estado da tela, header, mensagens, eventos e componentes principais.
frontend/index.html	CSS esta embutido em <code><style></code> ; estilos novos podem entrar aqui.

3. Ordem recomendada para implementar

1. Banco: definir persistencia por usuario.
2. Schemas: definir o formato do texto que entra e sai da API.
3. Router: criar endpoints autenticados de leitura e atualizacao.
4. main.py: registrar o router.
5. openrouter.py: permitir system prompt dinamico com fallback.
6. chat.py: buscar as instrucoes do usuario e repassar ao servico.
7. api.js: criar funcoes GET/PUT usando apiFetch.
8. App.jsx: criar estado, abrir configuracao, carregar, editar e salvar.
9. index.html: adicionar apenas o CSS necessario.
10. Testes manuais e testes existentes.

Regra de ouro

Depois de cada bloco funcionar, faça um commit pequeno. Se algo quebrar no frontend, voce ainda sabe que o backend estava funcional no commit anterior.

4. Backend - persistencia

4.1 models.py

Voce precisa de uma entidade que guarde um unico texto associado a um usuario. Conceitualmente, pense em uma tabela com chave primaria, user_id como chave estrangeira UNIQUE e um campo TEXT. O UNIQUE e o que transforma a relacao em 1:1 na pratica.

```
class <ModeloDeConfiguracao>(Base):
    __tablename__ = "<nome_da_tabela>"

    id = mapped_column(Integer, primary_key=True, ...)
    user_id = mapped_column(Integer, ForeignKey("users.id"), unique=True, nullable=False, ...)
    content = mapped_column(Text, ...)

    user = relationship("User", ...)
```

O trecho acima e apenas o formato generico do SQLAlchemy. Adapte nomes e estilo ao projeto. Nao e necessario guardar o system prompt padrao no banco: ausencia de registro pode significar "usar o padrao".

4.2 schemas

Crie schemas pequenos. Um request de atualizacao precisa receber o texto; uma resposta pode devolver o texto efetivamente salvo. Pydantic e FastAPI ja cuidam da validacao estrutural.

```
class <UpdateRequest>(BaseModel):
    content: str

class <Response>(BaseModel):
    content: str
```

4.3 router autenticado

O router precisa usar Depends(get_current_user) e Depends(get_db). O usuario deve ser identificado pelo token, nunca por um user_id enviado pelo navegador.

```
@router.get("<rota>")
def get_config(
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
```

```
):  
    # consultar usando current_user.id  
    # retornar conteudo salvo ou valor apropriado  
  
@router.put("<rota>")  
def update_config(  
    payload: <Schema>,  
    current_user: User = Depends(get_current_user),  
    db: Session = Depends(get_db),  
):  
    # procurar registro por current_user.id  
    # criar se nao existir; atualizar se existir  
    # db.commit(); db.refresh(...)
```

Isolamento

Nunca aceite um user_id do frontend para decidir de quem sao as instrucoes. A identidade correta ja existe em current_user.id.

5. Backend - OpenRouter e chat

5.1 openrouter.py

Hoje `_build_messages` sempre insere `_SYSTEM_PROMPT` como primeira mensagem. A mudança conceitual e permitir um prompt recebido por parametro e usar `_SYSTEM_PROMPT` quando esse parametro estiver ausente/vazio.

```
def _build_messages(*, user_message, history, system_prompt=None):
    resolved_prompt = system_prompt or _SYSTEM_PROMPT
    messages = [{"role": "system", "content": resolved_prompt}]
    ...
```

Depois, `generate_reply(...)` e `stream_reply(...)` precisam aceitar o mesmo parametro e repassa-lo a `_build_messages(...)`. Pense nisso como uma cadeia: `chat.py -> generate/stream -> _build_messages`.

```
generate_reply(..., system_prompt=<valor>)
stream_reply(..., system_prompt=<valor>)
```

5.2 chat.py

Este e o ponto em que voce tem `current_user` e `db` ao mesmo tempo. Portanto, e o lugar mais natural para buscar as instrucoes correspondentes ao usuario e transforma-las em um `system_prompt` opcional.

- No endpoint POST `/api/chat`: carregar instrucoes do `current_user` antes de chamar `generate_reply`.
- No endpoint POST `/api/chat/stream`: fazer a mesma coisa antes do `event_generator` ou capturar o valor em uma variavel que o generator consiga usar.
- Passar exatamente o mesmo `system prompt` para o fluxo normal e para o streaming.
- Se nao houver registro, repassar `None` (ou equivalente) para que `openrouter.py` use `_SYSTEM_PROMPT`.

Cuidado com o titulo automatico

`_generate_title(...)` tambem chama `generate_reply(...)`. Em geral, nao e desejavel que instrucoes pessoais do usuario alterem a tarefa interna de gerar titulo. Uma escolha simples e manter essa chamada usando o comportamento padrao, enquanto as respostas de chat usam as instrucoes personalizadas.

6. Frontend - API

6.1 api.js

`api.js` ja possui `apiFetch(...)` e `getAuthHeaders()`. Aproveite isso; nao reimplente token manualmente para a nova feature.

```
async function get<Config>() {
    const resp = await apiFetch("<rota>");
    return await resp.json();
}

async function save<Config>(content) {
    const resp = await apiFetch("<rota>", {
        method: "PUT",
        body: JSON.stringify({ content }),
    });
    return await resp.json();
}
```

Esse e um padrao generico de fetch. O importante e reutilizar `apiFetch` para manter `Content-Type`, `Authorization` e tratamento de erro consistentes.

7. Frontend - React sem panico

7.1 Estados que provavelmente serao necessarios

Em App.jsx, voce precisa representar tres coisas: se a interface de configuracao esta aberta, qual texto esta sendo editado e se existe carregamento/erro ao salvar.

```
const [settingsOpen, setSettingsOpen] = useState(false);
const [instructions, setInstructions] = useState("");
const [settingsBusy, setSettingsBusy] = useState(false);
const [settingsError, setSettingsError] = useState("");
```

Os nomes acima sao exemplos. O ponto e lembrar: useState guarda dados que mudam e cuja mudanca precisa redesenhar a tela.

7.2 Carregar ao abrir

Uma abordagem simples: ao clicar no botao de configuracoes, abrir o painel/modal e chamar a funcao GET da API. Outra opcao e useEffect dependente de settingsOpen. Escolha a que voce achar mais facil de depurar.

```
useEffect(() => {
  if (!settingsOpen) return;
  // chamar GET
  // setInstructions(resultado.content)
}, [settingsOpen]);
```

7.3 Textarea controlado

```
<textarea
  value={instructions}
  onChange={(e) => setInstructions(e.target.value)}
/>
```

Em React, value vem do estado e onChange atualiza o estado. Se esquecer onChange, o campo fica praticamente somente leitura.

7.4 Salvar

```
async function handleSave() {
  setSettingsBusy(true);
  setSettingsError("");
  try {
    // await save...(instructions)
    // fechar modal ou mostrar sucesso
  } catch (err) {
    // setSettingsError(...)
  } finally {
    setSettingsBusy(false);
  }
}
```

7.5 Onde colocar o botao

O App.jsx atual ja tem um header-right com email e botao Sair. Um botao de configuracao ao lado deles e uma alteracao pequena e facil de localizar. O CSS do projeto fica no <style> de frontend/index.html.

React em 20 segundos

useState = memoria da tela. useEffect = executa efeito quando dependencias mudam. value + onChange = input controlado. async/await + try/catch/finally = chamada de API com loading e erro.

8. CSS minimo

Nao gaste tempo tentando deixar perfeito. Voce so precisa de uma interface utilizavel: overlay opcional, caixa/modal, textarea e botoes. Reaproveite as variaveis CSS existentes como `--border`, `--text`, `--user-bubble` e `--composer-bg`.

```
.<overlay> {  
  position: fixed;  
  inset: 0;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  ...  
}  
  
.<modal> {  
  width: min(...);  
  background: var(--composer-bg);  
  border: 1px solid var(--border);  
  border-radius: ...;  
  ...  
}
```

9. Checklist de testes

- [] Aplicacao sobe sem erro de import.
- [] Usuario A abre configuracao e ve estado inicial adequado.
- [] Usuario A salva uma instrucao simples, por exemplo: "Responda sempre em portugues".
- [] Recarrega a pagina: a instrucao continua salva.
- [] Faz logout e login novamente: a instrucao continua salva.
- [] Envia mensagem: resposta segue a instrucao personalizada.
- [] Streaming continua funcionando.
- [] Usuario B nao enxerga a instrucao do usuario A.
- [] Usuario B sem configuracao continua usando o prompt padrao.
- [] Depois de alterar a instrucao, a proxima conversa usa o novo texto.
- [] Titulo automatico continua razoavel.
- [] Testes existentes continuam passando.

10. Se algo quebrar, procure nesta ordem

Sintoma	Primeiro lugar para olhar
404 na nova rota	Verifique <code>app.include_router(...)</code> em <code>main.py</code> e <code>prefix/path</code> do router.
401	Verifique se a chamada usa <code>apiFetch</code> e se ha <code>access_token</code> no <code>localStorage</code> .
422	Formato JSON nao bate com o schema Pydantic. Confira nome do campo.
500 ao salvar	Cheque <code>model</code> , <code>import</code> , <code>db.commit()</code> e se a tabela foi criada.
Salva mas some	Confira se voce fez <code>commit</code> e se consulta por <code>current_user.id</code> .
Usuario ve dado de outro	Filtro esta errado ou usa id vindo do cliente. Filtre exclusivamente por <code>current_user.id</code> .
Chat ignora configuracao	Confira a cadeia <code>chat.py -> generate/stream -> _build_messages</code> .
Funciona sem streaming, falha com streaming	Voce provavelmente alterou <code>generate_reply</code> mas esqueceu <code>stream_reply</code> ou o endpoint <code>/api/chat/stream</code> .
Textarea nao digita	Falta <code>onChange</code> ou o estado nao esta sendo atualizado.
Botao salva mas UI trava	Confira <code>finally { setSettingsBusy(false) }</code> .
CSS nao aparece	O CSS esta em <code>frontend/index.html</code> e pode haver cache; o projeto usa <code>headers no-cache</code> , mas recarregue a pagina.

11. Comandos uteis

```
# Rodar setup / aplicacao (Windows)
setup.bat
setup.bat run

# Testes (Windows)
.venv\Scripts\python.exe -m pytest

# Git
git status
git add <arquivos>
git commit -m "mensagem descritiva"
git push
```

12. Plano de emergencia - se faltar tempo

Priorize backend funcional antes de UI bonita. A ordem de valor e: persistencia correta -> endpoint autenticado -> prompt usado no streaming -> interface minima. Uma textarea simples e dois botoes valem mais do que gastar minutos com animacao ou layout.

Resumo de 15 segundos

BANCO -> API -> OPENROUTER -> CHAT/STREAM -> API.JS -> APP.JSX -> CSS -> TESTAR 2 USUARIOS. Se algo falhar, siga a cadeia de dados e nao tente consertar tudo ao mesmo tempo.